

KANTONSSCHULE REUSSBÜHL

ARBEITSJOURNAL

Entwicklung eines lizenzfreien Funkprotokolls zur Schliessung der Bandbreiten-Reichweiten-Lücke zwischen WLAN und LoRa

Valentin Seeholzer

K23a

Betreut durch:
Herr Daniel Zurmühle

Hinweis zur Führung: Dieses Journal wird chronologisch geführt. Es dokumentiert den Arbeitsprozess, Entscheidungen, Zweifel und Ergebnisse.

8. September 2026

Datum	Zeit	Thema & Ablage	Inhalt
14.10.25	1.0h	Projektstart & LoRa <i>Ablage</i>	Einrichten der OneNote-Struktur. Erste tiefergehende Recherche zur Faszination LoRa: Ich möchte verstehen, wie diese enorme Reichweite physikalisch möglich ist, während mein WLAN kaum in den Garten reicht. Schlussfolgerung: LoRa gewinnt an Reichweite, da das Signal über Zeit und Frequenz gespreizt wird. Der Nachteil ist jedoch, dass pro Zeiteinheit weniger Daten gesendet werden können im Vergleich zu anderen Modulationen wie FSK.
15.10.25	0.5h	Grundlagen Modulation <i>Ablage</i>	Verstehen der grundlegenden Modulationstechniken. Analyse von Trägersignalen, Amplitudenumtastung (ASK), Frequenzumtastung (FSK) und Phasenumtastung (PSK).
18.10.25	0.75h	Vertiefung CSS <i>Ablage</i>	Einarbeitung in Chirp Spread Spectrum (CSS). Ich habe verstanden, dass Up- und Down-Chirps orthogonal zueinander sind. Erste theoretische Überlegungen, wie ein "De-Chirping" funktionieren könnte. Experimentieren mit der Modulation von Sinus- und Cosinuswellen in GeoGebra, um die mathematische Natur von Chirps besser zu visualisieren.
15.11.25	0.25h	Literaturrecherche <i>Ablage</i>	Versuch, Standardwerke wie <i>Fundamentals of Spread Spectrum Modulations</i> (Springer) zu lesen. Erkenntnis: Die Werke sind für den Einstieg mathematisch zu komplex und liefern nicht die direkt umsetzbaren Praxis-Informationen, die ich suche.
19.11.25	0.25h	Analyse Symbolmapping <i>Ablage</i>	Analyse des LoRa-Symbolmappings. Ich habe mir visuell erarbeitet, wie Bits in Frequenzsprünge (Offsets) kodiert werden (2^{SF} Möglichkeiten).

Datum	Zeit	Thema & Ablage	Inhalt
24.11.25	1.0h	Effizienzsteigerung <i>Ablage</i>	Untersuchung von Möglichkeiten zur Steigerung der Datenübertragungsrate durch den Einsatz von Slope-Shift-Keying. Erkenntnis: Eine beliebige Variation der Chirp-Steigung verändert unweigerlich die genutzte Bandbreite und beeinträchtigt somit die Robustheit gegenüber Interferenzen. Die optimale Lösung besteht daher darin, die Steigung betragsmäßig konstant zu belassen und lediglich zwischen positivem und negativem Vorzeichen (Up- und Down-Chirps) zu unterscheiden. Dieser Ansatz liefert einen Datengewinn von exakt 1 Bit pro Symbol, was jedoch durch einen höheren Rechenaufwand bei der Demodulation erkauft wird.
25.11.25	0.5h	Fachgespräch P. Sies	Besprechung der möglichen Umsetzung des Themenvorschlages mit Frau Patricia Sies.
25.11.25	0.5h	Paper-Recherche	Recherche nach spezifischen Papers zur Effizienzsteigerung. <i>Dual-Mode CSS Modulation, Fine tuning of data rate enhances performance, On the Utility of Chirp Modulation for Digital Signaling.</i>
25.11.25	1.0h	FPGA-Umsetzung <i>Skizze: FPGA Xilinx Artix 7</i>	Recherche zur Umsetzung mittels FPGA (Zynq 7000 Serie, AD9361). Erstellung eines Blockdiagramms. Fazit: Teile scheinen sehr teuer und die Implementierung (VHDL) extrem aufwendig für den Zeitrahmen.

Datum	Zeit	Thema & Ablage	Inhalt
26.11.25	0.5h	Fachgespräch D. Zurmühle	Besprechung mit Herrn Daniel Zurmühle. Diskussion über die Komplexität und Umsetzbarkeit des Projekts sowie Anfrage bezüglich der Betreuung meiner Maturaarbeit.
26.11.25	1.0h	Wechsel auf SDR <i>Ablage</i>	Nach der Entscheidung gegen die Verwendung eines FPGAs wurde nach alternativen Wegen gesucht, um das entwickelte System später physisch testen zu können. In diesem Zuge erfolgte eine Recherche zum ADALM-Pluto SDR, insbesondere im Hinblick auf die Nutzung von libiio und der Python-Anbindung.
28.11.25	0.5h	Frequenzplanung <i>BAKOM RIR Dokumente</i>	Prüfung des BAKOM-Frequenzzuweisungsplans. Suche nach einer passenden Frequenz im 868-MHz-Band mit möglichst hoher EIRP und hohem Duty Cycle, die lizenzfrei (ohne Amateurfunklizenz) genutzt werden darf. In die engere Auswahl fielen dabei die Schnittstellenanforderungen RIR1003-10 (500 mW, 10% DC) und RIR1003-09 (500 mW, 2,5% DC).
29.11.25	0.5h	Fehlversuch: Redundanz <i>Ablage</i>	Versuch, die Richtung des <i>vorangegangenen</i> Chirps zu nutzen, um zusätzliche Datenbits pro Chirp zu übertragen. Ergebnis: Erfolglos. Die Datenrate liess sich nicht signifikant steigern, ohne die Fehleranfälligkeit massiv zu erhöhen.

Datum	Zeit	Thema & Ablage	Inhalt
30.11.25	1.25h	Fehlversuch: Offset <i>Ablage</i>	Versuch, einen 125-kHz-Chirp innerhalb eines 200-kHz-Kanals zu verschieben (Nutzung von <i>Frequency Slots</i>). Die Idee bestand darin, einen Mini-Chirp als Marker zur Erkennung des gewählten Slots einzusetzen. Ergebnis: Verworfen. Kurze Mini-Chirps weisen zu wenig Energie auf und gehen im Rauschen unter, lange bevor der Haupt-Chirp unlesbar wird. Dadurch limitiert der Mini-Chirp die Gesamtreichweite des Systems. Für diese reduzierte Reichweite wäre es deutlich effizienter, einfach den Spreading Factor zu senken.
02.12.25	2.0h	Mathematische Herleitung der LoRa-Modulation <i>Ablage</i>	Mathematische Herleitung des LoRa-Basisbandsignals. Berechnung der quadratischen Phasenfunktion durch Frequenzintegration sowie I/Q-Umformung mittels Eulerscher Formel. Definition der Chirp-Laufrichtung (Up/Down) über komplexe Konjugation.
06.12.25	0.5h	Coding: Simulation <i>Ablage</i>	Basis- Start der Programmierung einer ersten Basis-Simulation. Ablauf: Eine Bit-Nachricht wird mit Nullen auf die benötigte Symbollänge inklusive Richtungsbit aufgefüllt (Padding). Anschliessend erfolgt die Umwandlung in komplexe I/Q-Daten

Datum	Zeit	Thema & Ablage	Inhalt
07.12.25	1.75h	Coding: Simulation <i>Ablage</i>	Basis- Fortsetzung der Programmierung der Basis-Modulation und Implementierung der Empfänger-Logik. Erste erfolgreiche Signaldekodierung (De-Chirp) mittels FFT-Peak-Erkennung. Anschliessend Integration der Slope-Shift-Keying-Logik und Visualisierung im Spektrogramm. Dabei traten zunächst falsche Dekodierwerte bei Down-Chirps auf. Ursache: Die komplexe Konjugation (Invertierung) dreht auch die Shift-Richtung um. Nach Behebung dieses Fehlers verlief der Test erfolgreich; die unterschiedlichen Steigungen (Slopes) werden nun visuell und logisch korrekt verarbeitet.
08.12.25	0.75h	Recherche zu FEC <i>Ablage</i>	Effizienzsteigerungsversuch durch coding gain. Suche von Möglichkeiten. LoRa benutzt hamming codes für FEC diese sind zwar energieverbrauchs arm aber haben eine sehr schlechte Effizienz. Idee von Implementierung eines FEC wie in WLAN oder 5G. Es ergab sich, dass diese ein sogenanntes LDPC verwenden.
08.12.25	0.5h	Coding: Simulation	Basis- Implementierung der <code>add_noise</code> Funktion. Berechnung von SNR (Signal-to-Noise Ratio) bei komplexen I/Q-Signalen.
22.12.25	2.0h	Coding: Hinzufügen eines LDPC <i>Ablage</i>	Einarbeitung in LDPC (Low Density Parity Check) zur Fehlerkorrektur. Recherche nach einer geeigneten Generator-Matrix. Entscheidung, die IEEE 802.11n Matrix als Basis zu nutzen.
27.12.25	1.0h	Coding: Simulation	Basis- Integration des <code>BpDecoder</code> . Die komplette Simulation (Bits $\rightarrow$ LDPC $\rightarrow$ Chirp $\rightarrow$ Noise $\rightarrow$ Decode) läuft fehlerfrei durch.

Datum	Zeit	Thema & Ablage	Inhalt
29.12.25	2.0h	Recherche Halow	Recherche nach Alternativen für weite Distanzen mit hohem Datendurchsatz. Dabei Evaluation von Wi-Fi HaLow (IEEE 802.11ah). Dieses QAM-basierte System erreicht bis zu 30 Mbit/s über mehrere hundert Meter bis Kilometer, primär durch die Nutzung einer hohen Kanalbandbreite. Da diese Bandbreite im 868-MHz-Band in der Schweiz/EU rechtlich nicht zulässig ist, erfolgte eine gezielte Suche nach Frequenzbändern, die breitere Kanäle erlauben. Hierzu wurden die BAKOM-Richtlinien RIR1003-11 und RIR1003-12 geprüft, welche für Breitband-Datenübertragung vorgesehen sind. Erkenntnis: Mehr Bandbreite erlaubt kleineren Spreading Factor (SF) und damit einen höheren Durchsatz.(bei gleicher SNR)
05.01.26	1.25h	Antennen & MIMO	Antennen-Theorie: Yagi vs. Omni. Einarbeitung in dB, dBi, dBm, ERP Umrechnungen. Entscheidung, MIMO (2 Antennen) zur Reichweitensteigerung zu nutzen.
06.01.26	2.0h	Organisation	Strukturierung der Notizen, Anlegen des Overleaf-Dokuments für die schriftliche Arbeit.
10.01.26	0.75h	Coding: Simulation	live- Umbau der Datengenerierung auf dynamische Werte (<code>random_bits_input</code>), da statische Listen für umfassende Stresstests unzureichend sind. Architektur Anpassung auf Python-Generators (<code>yield</code>) zur Simulation kontinuierlicher Datenströme. Dies erleichtert die spätere Code Anpassung an den ADALM-Pluto SDR erheblich.

Datum	Zeit	Thema & Ablage	Inhalt
24.01.26	4h	Coding: Simulation live-	Implementierung der Daten-Segmentierung (<code>bits_to_chunk</code>), um den LDPC-Encoder mit der korrekten Blocklänge zu versorgen. Erstellung einer Endlosschleife für Stresstests inklusive automatischer Verifikation der empfangenen Bits. Zudem Integration eines SNR-Sweeps: Die Simulation reduziert schrittweise das Signal-Rausch-Verhältnis (SNR), bis Bitfehler auftreten, um die exakte Demodulationsgrenze des Systems zu ermitteln.
25.01.26	0.25h	Adaptiver Algorithmus	Konzeption des adaptiven Algorithmus. Erstellung einer Tabelle "Distanz vs. Mode". Definition der Umschaltpunkte zwischen verschiedenen Distanzen und Datengeschwindigkeit. Ausarbeitung des "Graceful Degradation Verhaltens: Das System soll bei schlechtem Empfang nicht abbrechen, sondern die Qualität reduzieren.
03.02.26	0.75h	Fachgespräch D. Zurmühle <i>Gesprächsnotizen</i>	Besprechung mit Herr Daniel Zurmühle über das weitere Vorgehen (Hardware-Aufbau), Kommunikationswege und Abschluss des Maturaarbeitsvertrages.
04.02.26	1.0h	Journalführung	Übertragung der projektbegleitenden Notizen und systematische Erfassung des Arbeitsjournals in LaTeX.
08.02.26	1.25h	Theoretische Grundlagen	Recherche zur Berechnung des Link-Budgets: Untersuchung der Freiraumdämpfung (Free-Space Path Loss, FSPL) sowie des Grundrauschens (Noise Floor) und der spezifischen Rauschzahl (Noise Figure) des ADALM-Pluto SDRs.

Datum	Zeit	Thema & Ablage	Inhalt
11.02.26	1.0h	Recherche zu PSK	Evaluierung zusätzlicher Modulationsarten zur Steigerung der Datenrate. FSK und ASK kommen aufgrund ihrer limitierten Reichweite und Störanfälligkeit nicht infrage. PSK (Phase Shift Keying) erweist sich hingegen als vielversprechend für einen hybriden Ansatz.
12.02.26	0.25h	Anfrage an sourcesdr lab Datenblätter	Verfassen einer englischsprachigen E-Mail an den Support von OpenSourceSDR Lab. Ziel war es, ein detailliertes Datenblatt sowie ein technisches Referenzhandbuch für das "New 7020-SDR Professional Edition Development Board" anzufordern.
19.02.26	0.50h	Antwort erhalten von Opensourcesdr lab	Rückmeldung vom Support-Mitarbeiter Kevin erhalten. Die E-Mail beinhaltet einen Download-Link (workupload) zu einem Archiv mit dem Namen "7020_AD936X_SDR". Herunterladen der Dateien und kurze erste Sichtung der bereitgestellten Dokumentationen für die weitere Planung mit der Hardware.
22.02.26	2.0h	Journalführung	Abschliessende Formatierung des Arbeitsjournals in LaTeX sowie Einrichtung eines GitHub-Repositories zur Ablage des Codes.
23.02.26	0.5h	Coding Kommentare	Anpassung der Kommentare in den Skripten <code>Basis_Simulation</code> und <code>Live_Simulation</code> gemäss den Vorgaben des Dokuments <i>Coding Standards</i> (bereitgestellt von Herrn Zurmühle).
26.02.26	1h	Rechtliche Abklärung <i>E-Mail an BAKOM / Projektbeschreibung</i>	Kontaktaufnahme mit dem BAKOM bezüglich der rechtlichen Rahmenbedingungen für den SDR-Sendebetrieb im 2.4-GHz-Band. Anfrage zur Notwendigkeit und den hohen Kosten einer Versuchskonzession für schulische Projekte sowie Einreichen des detaillierten Projektbeschriebs zur Prüfung.

Datum	Zeit	Thema & Ablage	Inhalt
8.03.26	0.5h	Disposition	Schreiben einer Kurzen disposition mit Fragestellung, These, vorläufiges Inhaltsverzeichnis, bibliographie und grober Zeitplan.
10.03.26	0.5h	PSK Spread-Spectrum	Entwicklung eines Ansatzes über Implementierung einer Phasen verschiebung innerhalb der Jeweiligen Symbols(chirp) mit QPSK um 2 zusätzliche bits zu erhalten.
11.03.26	0.5h	Status-Check der BAKOM-Anfrage.	Bisher noch keine Eingangsbestätigung erhalten. Die Feldtests bleiben bis zur rechtlichen Klärung sistiert, um die regulatorische Konformität sicherzustellen. Fokus liegt aktuell auf der weiteren Optimierung des Python-Codes im Simulations-Modus.
13.03.26	0.25h	Telefonische Rückmeldung vom BAKOM bezüglich Konzessionspflicht.	Das BAKOM hat telefonisch bestätigt, dass für das Projekt keine spezifische Konzession notwendig ist. Da der Versuchsaufbau im 2.4ghz erfolgt, die Sendeleistung unter 100 mW bleibt und das rir1010-01 eingehalten wird, gilt der Betrieb als konzessionsfrei. Konzessionen bei der Benutzung von Sdrs sind erst notwendig wenn diese an Dritte weitergegeben oder verkauft werden.
16.03.26	1.0h	Besprechung Disposition	Besprechung der Disposition mit Herrn Daniel Zurmühle. Themen waren die Planung des weiteren Vorgehens, der Zeitplan und die Materialbeschaffung. Abschliessend erfolgte eine Terminvereinbarung für die Anforderungsanalyse der Programme sowie die Besprechung der Kriterien.

Datum	Zeit	Thema & Ablage	Inhalt
17.03.26	0.25h	Sponsoring-Anfrage an OpenSourceSDRLab	Verfassen und Versenden einer Sponsoring-Anfrage an OpenSourceSDRLab für das SDR-Board mit den Spezifikationen Zynq-7020, AD9363 und PGA102+. Kurze Zeit später eine erste Rückmeldung erhalten: Das Anliegen wurde zur weiteren Prüfung an die interne Sponsoring-Abteilung weitergeleitet.
18.03.26	2h	Maturaarbeit verfassen: Theorie	Verfassen des Theorieteils zu den grundlegenden digitalen Modulationsarten (ASK, FSK und PSK) unter Einbezug der Fachquelle pysdr.org. Zudem wurde ein Vergleich dieser klassischen Verfahren mit der Chirp-Spread-Spectrum-Modulation (CSS) von LoRa ausgearbeitet.
24.03.26	1.5h	Coding: Simulation live-	Implementierung der Phasenmodulation für QPSK. Dies umfasste die Signalgenerierung mit 45°-QPSK-Offset, das De-Chirping via komplexer FFT zur Phasenrückgewinnung sowie die Schnittstelle zum LDPC-Decoder. Ein erster Systemtest ergab eine fehlerhafte Bitübertragung. Als nächster Schritt ist ein systematisches Debugging zur Fehlerisolierung geplant.

Datum	Zeit	Thema & Ablage	Inhalt
24.03.26	2h	Coding: Simulation und Live-Debugging	Debugging der Pipeline mittels Print-Ausgaben ohne LDPC-Decodierung. Die Analyse ergab, dass die LoRa- und Richtungs-Werte bei 6 dB SNR korrekt empfangen werden, der Fehler jedoch die QPSK-Phase betrifft (zuvor war nur ca. 1 von 5 Chirps bezüglich der Phase korrekt). Als Ursache wurden zwei Effekte bei der Signalgenerierung identifiziert: Die zyklische Zeitverschiebung (<code>np.roll</code>) für das LoRa-Symbol erzeugte einen unbeabsichtigten, variablen Phasen-Offset. Ausserdem war die Berechnung der QPSK-Phase im Code ungenau. Das Problem wurde gelöst, indem der unerwünschte Phasenversatz nun schon im Sender ausgeglichen wird und für die Phasenverschiebung der korrekte Befehl <code>np.exp()</code> genutzt wird. Dadurch werden die QPSK-Werte jetzt nahezu fehlerfrei empfangen.
29.03.26	3h	Performance-Analyse und Hardware-Entscheidung	Im Rahmen der Vorbereitungen für das Decoding wurde festgestellt, dass die benötigte Verarbeitungsgeschwindigkeit in herkömmlichen Programmiersprachen wie C oder Python nicht erreicht werden kann. Da die Echtzeitanforderungen des Systems eine sehr geringe Latenz erfordern, wurde entschieden, rechenintensive Operationen wie die FFT (Fast Fourier Transformation) und einfache Multiplikationen direkt auf das FPGA auszulagern. Dieser Schritt ist notwendig, um die parallele Verarbeitungsleistung der Hardware zu nutzen und den Flaschenhals der sequentiellen Software-Ausführung zu umgehen.

Datum	Zeit	Thema & Ablage	Inhalt
02.04.26	4h	Modul-Generierung und KI-Unterstützung	Für den Versuch der Hardware-Portierung der LoRa-QPSK-Logik wurde die KI Claude eingesetzt, um die komplexen Algorithmen aus Python in Verilog (.v-Dateien) zu übersetzen. Die KI generierte dabei die Grundstruktur der Module sowie die spezifische Logik für die Hardware-Multiplikatoren. Es muss explizit festgehalten werden, dass der erzeugte Verilog-Code in seiner Gesamtheit und exakten Syntax nicht vollständig durchdrungen wurde; das Verständnis beschränkt sich auf Bruchteile und die allgemeine Logikstruktur. Die Nutzung der KI diene hierbei als Brücke, um die theoretischen Konzepte trotz begrenzter HDL-Kenntnisse in Hardware-Beschreibungen zu überführen.
05.04.26	5h	Systemintegration in Vivado	Der letzte Schritt dieses Versuchs umfasste die Einbindung der generierten Verilog-Module in das vom Hersteller bereitgestellte Referenzdesign innerhalb von Xilinx Vivado. Unter Anleitung der KI wurden die Schnittstellen der neuen Module mit dem bestehenden System-IP (Intellectual Property) des Herstellers verdrahtet. Der Fokus lag hierbei primär auf der korrekten strukturellen Verbindung im Blockdesign, um einen konsistenten Datenfluss zwischen dem Funkmodul und der Verarbeitungslogik sicherzustellen. Ziel war es, die technische Machbarkeit der Integration zu prüfen, ohne jedes Detail der zugrunde liegenden Verilog-Syntax tiefgreifend zu analysieren.

Datum	Zeit	Thema & Ablage	Inhalt
09.04.26	1.5h	Gehäusedesign in Fusion 360	In dieser Arbeitsphase wurde das Gehäuse (Case) für die Software Defined Radios (SDRs) entworfen. Für die Modellierung kam die CAD-Software Autodesk Fusion 360 zum Einsatz. Ziel war es, ein passgenaues Gehäuse zu konstruieren, welches die Platinen sicher aufnimmt und gleichzeitig die nötigen Aussparungen für Anschlüsse und Antennen bietet.
12.04.26	2h	3D-Druck und Hardware-Montage	Das zuvor in Fusion 360 erstellte Gehäusedesign wurde im 3D-Drucker gefertigt. Anschliessend erfolgte der physische Aufbau der Hardware: Die SDR-Platinen wurden in das gedruckte Case eingesetzt, passend verschraubt und die externe Verkabelung vorbereitet.
13.04.26	3h	Netzwerk-Konfiguration und Inbetriebnahme	Der erste Versuch, eines der SDR-Boards mit dem PC zu verbinden, erwies sich als herausfordernd, da die offizielle Dokumentation des Herstellers ausschliesslich auf Chinesisch vorlag. Mithilfe der KI Gemini wurden die relevanten Textpassagen übersetzt. Nach einer intensiven Testphase konnte schliesslich eine stabile Kommunikationsverbindung zwischen dem PC und dem SDR via Ethernet hergestellt werden.

Datum	Zeit	Thema & Ablage	Inhalt
13.04.26	1h	Treiberprobleme und erfolgreicher Test mit libiio	Bei der anschliessenden Nutzung der offiziellen SDR-Software des Herstellers traten Kompatibilitätsprobleme auf: Das Board wurde aufgrund fehlender oder nicht erkannter Treiber nicht unterstützt. Als Alternative wurde ein von Gemini generiertes Testskript herangezogen, welches direkt auf der libiio-Bibliothek basiert. Dieser Ansatz war erfolgreich: Das Skript konnte Daten vom SDR auslesen, was die grundsätzliche Funktionsfähigkeit der Hardware und der Kommunikationsschnittstelle bestätigte.
14.04.26	3h	Anpassung der Linux-Netzwerkconfiguration	Das ARM-basierte SDR-Board läuft auf einem Linux-System. Die vom Hersteller empfohlene Zuweisung der IP-Adresse via <code>ifconfig eth0</code> funktioniert zwar, geht aber bei jedem Stromverlust verloren. Der Versuch, die IP-Adresse dauerhaft in den Konfigurationsdateien des Systems (<i>config files</i>) festzulegen, war erfolglos, da die korrekte Datei für diese spezifische Linux-Version nicht ausfindig gemacht werden konnte. Um den Projektfortschritt nicht weiter zu verzögern, wurde entschieden, die IP-Adresse stattdessen vor jedem Testlauf manuell über eine serielle USB-Verbindung zu definieren.
15.04.26	1.5h	Erstellung der Anforderungsanalyse	In dieser Phase wurde eine detaillierte Anforderungsanalyse für das gesamte Projekt verfasst. Dabei wurden die Projektziele präzisiert und die Anforderungen in funktionale Grundfunktionen (Must-have), erweiterte Funktionen (Should-have) und optionale Zusatzfunktionen (Nice-to-have) unterteilt. Ein besonderer Fokus lag auf der Definition der technischen Schnittstellen zwischen der Python-Software und der Zynq-Hardware.

Datum	Zeit	Thema & Ablage	Inhalt
16.04.26	0.5h	Erstellung eines Spektrumanalysators (Python)	Aufgrund fehlender Treiber und Fehlfunktionen beim Spektrumanalysator des Herstellers wurde eine eigene Alternative in Python umgesetzt. Da dieses Programm nur als Werkzeug für die Messungen dient und nicht Teil der eigentlichen Projektentwicklung ist, wurde der Quellcode zeitsparend mithilfe von Gemini generiert und anschliessend durch mich validiert.
17.04.26	1h	Implementierung des SDR-Testcodes in Python	Aufbau des Basis-Skripts für die Tests mit der echten SDR-Hardware. Die funktionierenden mathematischen Bausteine aus <code>End_simulation.py</code> wurden erfolgreich in die neue Datei übernommen. Danach wurde die Hardware-Schnittstelle über die Analog Devices Bibliothek in Python integriert. Abschließend wurden der Initialisierungsprozess für das SDR-Board und die Struktur der Hauptschleife (Main Loop) programmiert.
24.04.26	2.5h	Programmierung des Pluto_test.py in Python	Verfeinerung der Hauptschleife. Aktuell muss man noch zwischen dem Senden und Empfangspfad wechseln, beides gleichzeitig funktioniert noch nicht. Ausserdem wurde eine Präambel implementiert und die dazugehörige Logik für das Senden und Empfangen programmiert.

Datum	Zeit	Thema & Ablage	Inhalt
25.04.26	1h	Erster Hardware-Empfangstest und Fehleranalyse	Durchführung eines ersten Empfangstests über die Luft. Der Versuch schlug fehl: Obwohl das gesendete Signal des PlutoSDRs im Spektrumanalysator deutlich sichtbar war, konnte das Empfangsskript keine gültigen Daten detektieren. Als primäre Fehlerquellen kommen ein starker Trägerfrequenzversatz (Carrier Frequency Offset, CFO) zwischen den lokalen Oszillatoren der Plutos oder eine generelle Fehlkonfiguration der Hardware beziehungsweise Software infrage. Ein systematisches Debugging ist als Nächstes erforderlich.
27.04.26	1h	Prüfung der Anforderungsanalyse	Gemeinsame Besprechung und Evaluierung der Anforderungsanalyse. Die festgelegten Projektziele und technischen Anforderungen an die geplante Software wurden diskutiert.
30.04.26	1.5h	CFO-Logik I: Präambel-Erkennung und Binning	Beginn der Programmierung zur Korrektur des Trägerfrequenzversatzes (CFO). Zunächst wurde ein Algorithmus implementiert, der den detektierten Wert der Präambel-Chirps präzise in FFT-Bins erfasst. Diese abweichenden Bin-Werte dienen als rechnerische Grundlage, um im nächsten Schritt den zeitlichen Offset sowie den genauen Frequenzversatz zu quantifizieren.
02.05.26	2h	CFO-Logik II: Zeitliche Synchronisation (Fine Timing)	Aufbauend auf den erfassten Bin-Werten der Präambel wurde die Berechnung für das exakte Timing programmiert. Der Code berechnet nun die genaue zeitliche Verschiebung und passt das Empfangsfenster (Array-Index) für das ankommende Signal an. Durch diese zeitliche Verschiebung sitzt das Auslesefenster nun perfekt auf den Symbolen, wodurch die reinen LoRa-Werte wieder korrekt decodiert werden.

Datum	Zeit	Thema & Ablage	Inhalt
03.05.26	3h	CFO-Logik III: Phasen-Rückrotation für QPSK	Abschluss der CFO-Kompensation durch die Implementierung der Phasenkorrektur. Da sich die Phase der Funkwelle durch den verbleibenden Frequenzversatz kontinuierlich weiterdreht, gehen die QPSK-Werte ohne Korrektur verloren. Es wurde eine mathematische Rückrotation programmiert, die das Signal virtuell um den berechneten Offset zurückdreht, um auch die Phasenwerte (QPSK) fehlerfrei extrahieren zu können.
05.05.26	1h	Administrative Planung des Schreibprozesses	Anfertigung einer Kapitelübersicht, die einen administrativen Überblick über noch zu schreibende Textteile verschafft. Der Arbeitsfortschritt wird dabei in die drei Status 'Ausstehend', 'Check-up' und 'Erledigt' unterteilt. Ausserdem wurde ein erstes Konzept für eine sinnvolle Anordnung der Kapitel und Unterkapitel im Bereich 'Theoretische Grundlagen' entworfen.

Datum	Zeit	Thema & Ablage	Inhalt
09.05.26	1h	Ausschluss von Hardware-Fehlern und CFO-Analyse	Um Hardware-Defekte oder interne Eigenschwingungen des SDRs als Fehlerquelle für die Phantom-Signale auszuschließen, wurden Tests ohne angeschlossene Antenne (Terminierung) durchgeführt. Da hierbei keine Detektionen stattfanden, ist das Phänomen eindeutig auf die Software-Signalverarbeitung oder externe Einstrahlungen zurückzuführen. Weiterhin wurde die Carrier Frequency Offset (CFO) Korrektur entfernt, um zu prüfen, ob der Algorithmus thermisches Rauschen überkompensiert und dadurch Fehlalarme generiert. Da der verwendete PlutoSDR über einen temperaturkompensierten Oszillator (TC-XO) mit einer Stabilität von 0,5 ppm verfügt, resultiert dies bei einer Trägerfrequenz von 2,45 GHz in einem maximalen Frequenzversatz von lediglich ca. 1,2 kHz. Dieser minimale Drift ist für die verwendete LoRa-Bandbreite physikalisch vernachlässigbar und erfordert keine aggressive softwareseitige Kompensation.

Datum	Zeit	Thema & Ablage	Inhalt
09.05.26	2h	Analyse des ISM-Bandes, AGC-Effekte und Rechenlatenz	Trotz der Anpassungen verzeichnete das System weiterhin Falsche preambles. Eine spektrale Analyse zeigte, dass das genutzte ISM-Band bei 2,45 GHz Interferenzen (z. B. durch Mikrowellenöfen) ausgesetzt ist. Obwohl die umliegende WLAN-Infrastruktur primär die nicht überlappenden Kanäle 1, 6 und 11 nutzt und den Mittenbereich um 2,45 GHz tendenziell meidet, reicht die Störenergie für Fehlalarme in der Kreuzkorrelation aus. Als Gegenmaßnahme wurde die Empfangsverstärkung des SDRs auf eine automatische Regelung (AGC, Modus "Fast Attack") umgestellt, um Pegelspitzen dynamisch abzufangen. Bei der Evaluation zeigte sich jedoch ein systematischer Nachteil: Abrupte Gain-Wechsel der Hardware während einer laufenden Aufzeichnung erzeugen starke Amplitudensprünge im Signal, welche die Korrelation fälschlicherweise triggern könnten. Als Hauptursache für die generelle Unzuverlässigkeit kristallisierte sich letztlich die Rechenlatenz heraus: Die in Python implementierte Kreuzkorrelation (Faltung / Convolution) beansprucht zu viel Ausführungszeit, wodurch Signalpakete auf Software-Ebene zerschnitten oder verpasst werden.

Datum	Zeit	Thema & Ablage	Inhalt
14.05.26	3h	Rauschpegel-Kalibrierung und TDMA-Implementierung	Um empfangene Signale zuverlässiger vom Hintergrundrauschen und anderen Störsignalen unterscheiden zu können, wurde eine dynamische Rauschpegel-Kalibrierung implementiert. Beim Programmstart führt das System nun automatisch 20 Messungen durch, um den durchschnittlichen Rauschboden des Kanals zu ermitteln. Bei den anschließenden Tests stellte sich heraus, dass die rechenintensive Kreuzkorrelation zu viel Zeit beansprucht, um durchgehend in Echtzeit zu laufen. Als Lösung wurde ein TDMA-Ansatz (Time Division Multiple Access) implementiert, der die Sende- und Empfangszeiten in feste Slots unterteilt. Dabei wurden 60 % der Slot-Zeit explizit für die Decodierung reserviert. Die grundlegenden Funktionen für das einmalige Senden und Empfangen blieben unverändert.
14.05.26	1h	Fehlersuche Signalerkennung	Erste Tests zeigten, dass der Empfänger die gesendeten Signale weiterhin nicht zuverlässig erkennt. Zur Fehleranalyse wurde ein Spektrumanalysator mit Wasserfalldiagramm eingesetzt. Es kamen Zweifel auf, ob die Kombination aus Python-Skript und Hardware solch extrem kurze Signale überhaupt erfassen kann. Ein einzelner Chirp besteht bei den gewählten Parametern nur aus 32 Samples und hat eine physikalische Dauer von lediglich 1,6 Mikrosekunden.

Datum	Zeit	Thema & Ablage	Inhalt
15.05.26	1h	Sichtbarmachung des Signals im Spektrum	Um zu verifizieren, dass die Hardware physisch sendet, wurde versucht, ein Signal auf dem Wasserfalldiagramm sichtbar zu machen. Die reguläre Payload von 324 Bits entspricht nur 47 Chirps (ca. 76 Mikrosekunden), was für die visuelle Darstellung viel zu kurz ist. Als Lösungsansatz wurde die Präambel massiv verlängert (auf über 5000 Symbole). Im Spektrogramm zeigten sich daraufhin punktuelle Ausschläge statt der für Chirps typischen diagonalen Linien. Dies erklärt sich durch den diskreten Abtastprozess (die FFT-Fenster des Analysators): Da alle Präambel-Symbole identisch sind (Basis-Chirp ohne zyklische Verschiebung), erfasst die abschnittsweise Analyse den Frequenz-Sweep immer als isolierte Punkte an denselben Frequenzstellen, wodurch keine durchgehende Linie gezeichnet wird.

Datum	Zeit	Thema & Ablage	Inhalt
15.05.26	1h	Deaktivierung von TDMA und Entdeckung des Phasen-Drifts	Nachdem andere Fehlerquellen minimiert wurden, fiel der Verdacht auf das TDMA-Timing. Offenbar reichte die reservierte Zeit im Slot nicht aus, wodurch der Code die Pakete verpasste. Um dies zu umgehen, wurde die Empfangslogik auf eine kontinuierliche Aufzeichnung umgebaut: Das System zeichnet den Puffer nun ununterbrochen auf, bis der Vorgang manuell gestoppt wird, und führt die Decodierung erst im Anschluss durch. Mit dieser Methode konnte das Signal endlich erfolgreich gefunden und synchronisiert werden! Allerdings zeigten sich bei der Demodulation systematische Bitfehler: In jedem 7-Bit-Symbolblock wird das letzte Bit, welches die QPSK-Phase repräsentiert, periodisch falsch decodiert. Dies ist ein klares Symptom für einen Hardware-Frequenzdrift (Carrier Frequency Offset). Die Oszillatoren driften leicht auseinander, wodurch die Phase des Signals über die Zeit rotiert und die QPSK-Werte verfälscht werden.
16.05.26	0.5h	Bestellung GPS-Module	Da die Fehler im TDMA-System auf Ungenauigkeiten in der Zeitsynchronisation von Windows zurückzuführen sein könnten, wurden zwei GPS-Module (u-blox NEO-6M) bestellt. Ziel ist es, das System auf die hochpräzise Satellitenzeit zu synchronisieren, um diese Schwachstelle zu eliminieren. Die Synchronisation soll künftig über den PPS-Pin (Pulse Per Second) des Moduls erfolgen. Das Signal wird von einem ESP32 ausgelesen und über eine serielle Verbindung an den Laptop weitergeleitet.

Datum	Zeit	Thema & Ablage	Inhalt
20.05.26	1h	Fertigstellung der Gehäuse	Die Gehäuse beider Geräte wurden fertiggestellt. Die passenden Antennen-Verlängerungskabel wurden mithilfe von rechtwinkligen SMA-Adaptern angebracht. Zusätzlich wurde eine Abdeckung für die Ethernet- und USB-Ports designt und gedruckt. Abschliessend wurde der 5V-4020-Lüfter eingesetzt und verlötet.
21.05.26	1h	Verfassen des Theorieteils (Maturaarbeit)	Das Unterkapitel zur Hochfrequenztechnik wurde fertiggestellt. Darin behandelt werden logarithmische Pegelmasse, Antennengewinn, EIRP, Freiraumdämpfung, Noise Floor, Noise Figure sowie die Berechnung des Link Budgets.
25.05.26	1h	Verfassen des Theorieteils (Maturaarbeit)	Die theoretischen Abschnitte zu den Grundlagen der Fehlerkorrektur, zu Software Defined Radio (SDR) sowie zu den regulatorischen Rahmenbedingungen wurden verfasst.
27.05.26	1h	Verfassen der Maturaarbeit	Im Kapitel "Material und Methoden" wurde der Abschnitt zur SDR-Plattform innerhalb des Unterkapitels "Hardware-Setup" verfasst und anhand von Diagrammen veranschaulicht.
30.05.26	4h	Hardware-Integration der GPS-Module	Die GPS-Module wurden mit einem ESP32 verlötet und am Gehäuse verschraubt. Da das Modul über keinen herausgeführten PPS-Pin verfügte, wurde ein feiner Draht direkt an den PPS-Ausgang (an der Leiterbahn zur Status-LED) angelötet. Nach längerem Debugging fiel auf, dass die LED des ersten Moduls nicht leuchtete. Ein Kreuztest mit dem zweiten Modul bestätigte schliesslich: Das erste Modul sowie dessen Antenne wiesen einen Hardware-Defekt auf.

Datum	Zeit	Thema & Ablage	Inhalt
01.06.26	0.5h	Materialbestellung	Da für jedes Sende-Empfangs-Gerät ein eigenes GPS-Modul erforderlich ist, wurde ein weiteres u-blox NEO-6M bestellt. Zeitgleich wurde als Upgrade-Alternative das Modul ATGM336H evaluiert, welches durch die zusätzliche Unterstützung von GLONASS und BeiDou (BDS) eine deutliche Verbesserung zum NEO-6M darstellt und wurde somit auch mit bestellt. Ausserdem wurden Kupferklebeband für die spätere EMI-Abschirmung des Gehäuses sowie Kapton-Tape zur elektrischen Isolierung bestellt.
12.06.26	0.5h	Eingang Material <i>Ablage</i>	Ankunft und Vollständigkeitskontrolle der bestellten Komponenten (ATGM336H GPS-Module, Kupferfolie, Kapton-Tape). Kurze Überprüfung der GPS-Module auf äusserliche Unversehrtheit zur Vorbereitung des Einbaus.
22.06.26	0.5h	EMI-Abschirmung Gehäuse <i>Ablage</i>	Anbringen der Kupferfolie an den Innenseiten der 3D-gedruckten SDR-Gehäuse zur Abschirmung elektromagnetischer Interferenzen (Faradayscher Käfig). Anschliessende Isolierung mit Kapton-Tape zur Vermeidung von elektrischen Kurzschlüssen mit der Platine.
29.06.26	6.0h	Maturaarbeitswoche: LaTeX & Latenzanalyse <i>Ablage</i>	Einrichten der lokalen LaTeX-Umgebung in VS Code, da Overleaf an Leistungsgrenzen stiess. Weiterarbeiten am Dokument. Systematische Fehleranalyse mittels Python-Testskripts: Ein Puffer-Überlauf (Buffer Overrun) konnte ausgeschlossen werden; der Fokus verlagert sich auf Latenz- und Timing-Probleme.

Datum	Zeit	Thema & Ablage	Inhalt
30.06.26	6.0h	Maturaarbeitswoche: Literaturrecherche <i>Werner: Nachrichtentechnik</i>	Vertiefte Einarbeitung in die Fachliteratur (<i>M. Werner: Nachrichtentechnik</i>, 7. Aufl.) zur wissenschaftlichen Fundierung von Kapitel 4 (Theoretische Grundlagen). Schwerpunkte: • Erarbeitung der mathematischen Zusammenhänge von thermischem Rauschen ($P_R = k \cdot T \cdot B$), Noise Floor und dem Shannon-Hartley-Gesetz. • Systematischer Vergleich klassischer digitaler Modulationsarten (ASK/OOK, FSK, PSK) mit Chirp Spread Spectrum (CSS). • Einbindung der Zitate in die LaTeX-Struktur.
01.07.26	5.0h	Maturaarbeitswoche: Feldtests & Fehleranalyse <i>Ablage</i>	Praktische Testreihen unter verschiedenen Umgebungsbedingungen (rauscharmer Keller, offener Sportplatz, urbanes Gebiet, Schulzimmer). Erkenntnis: Die Phasenrotation stellt zwar eine Herausforderung dar, das primäre Nadelöhr ist jedoch die unzuverlässige Paketanfangserkennung (Preamble Detection).
02.07.26	8.0h	Maturaarbeitswoche: Parameteroptimierung <i>Ablage</i>	Durchführung weiterer Testreihen mit variierenden Parametern (Bandbreite, Abtastrate usw.). Parallel dazu Weiterarbeit am Manuskript der Maturaarbeit (Strukturierung und Ausformulierung der Kapitel <i>Theoretische Grundlagen</i> sowie <i>Material und Methoden</i>).

Datum	Zeit	Thema & Ablage	Inhalt
03.07.26	4.0h	Maturaarbeitswoche: Expertenanfrage <i>E-Mail HSLU</i>	Fokus auf die Lösungsfindung für das Synchronisationsproblem. Verfassen und Versenden einer E-Mail an Thomas Hunziker (Fachexperte der HSLU) zur externen Evaluation meiner Vermutungen bezüglich Carrier Frequency Offset (CFO) und Phasenrotation. Zusammenfassung und Dokumentation der Erkenntnisse aus der Intensivwoche.
27.07.26	2.0h	Brute-Force-Synchronisation <i>Ablage</i>	Konzeption einer zweistufigen Brute-Force-Suchmethode zur robusteren Präambel- und Paketerkennung im I/Q-Datenstrom. Fazit: Die Methode verspricht eine deutlich höhere Erkennungsrate bei schwachen Signalen, erfordert jedoch eine höhere Rechenleistung.
28.07.26	2.0h	DQPSK-Modulation <i>Ablage</i>	Einlesen in die differentielle QPSK (DQPSK). Umstellung der Phasenübertragung von absoluter auf differentielle Phasenmodulation, um Instabilitäten durch Phasenrotationen zu beheben. Ergebnis: Nach der Implementierung zeigt sich eine drastisch verbesserte Bit-Fehler-Stabilität bei der Phasenauswertung.
29.07.26	2.0h	GPS-Integration (ATGM336H) <i>Hardware</i>	Einbau und Verlöten der neuen ATGM336H-Module mit dem ESP32. Das Abgreifen des PPS-Signals für das Hardware-Timing funktioniert signifikant zuverlässiger als beim defekten Vorgängermodell. Erste Tests zeigen einen extrem schnellen Satelliten-Fix dank kombinierter GPS/GLONASS/BDS-Unterstützung.

Datum	Zeit	Thema & Ablage	Inhalt
30.07.26	3.0h	Automatisierte Testskripte <i>Testing</i>	Programmierung eines automatisierten Testskripts zur Auswertung der neuen, zweistufigen Brute-Force-Synchronisation. Das Skript liest aufgezeichnete I/Q-Dumps ein, iteriert über die Korrelationskandidaten und verifiziert die Payload. Die Kombination aus Brute-Force-Detektion und DQPSK zeigt eine massiv höhere Stabilität bei der Dekodierung.
02.08.26	0.5h	LaTeX-Überarbeitung Code-Design <i>LaTeX</i>	& Überarbeitung des LaTeX-Manuskripts und Behebung kritischer Kompilierungsfehler (u. a. Maskierung von Unterstrichen in Variablennamen wie <code>qpsk_wert</code>). Aktualisierung der Code-Highlighting-Konfiguration (<code>listings</code>) für eine ansprechende, gut lesbare Darstellung der Python-Skripte.
04.08.26	4.0h	TDMA-Verschränkung Burst-Modus <i>Ablage / Code</i>	& Das Problem der Puffer-Überläufe in Python konnte ich durch eine Anpassung im Code stark verbessern, auch wenn die Latenz nicht ganz verschwindet. Lösungsansatz: Ich habe eine GPS-synchronisierte TDMA-Logik im Burst-Modus eingebaut. Ein Gerät zeichnet das Signal jetzt nur kurz auf (z. B. 3 Pakete am Stück) und nutzt die restliche Zeit im Slot rein für die rechenintensive Dekodierung (FFT, LDPC). Das andere Gerät wartet in dieser Zeit einfach. So kommt das strenge Timing der Hardware nicht mehr so schnell aus dem Takt.

Datum	Zeit	Thema & Ablage	Inhalt
06.08.26	1.5h	Laptops im Vergleich & BER <i>Messprotokolle</i>	Ich habe das neue TDMA-System auf zwei verschiedenen Laptops (HP Omnibook Ultra vs. HP Spectre) getestet. Erkenntnis: Auf dem leistungsstarken Omnibook liegt die Bitfehlerrate (BER) bei genau 0.00 %, während das ältere Spectre 0.84 % BER hat. Das zeigt mir ganz klar, dass mein Funkmodem an sich fehlerfrei arbeitet. Python stösst auf schwächeren Prozessoren aber einfach an seine Grenzen, wodurch ab und zu Samples verloren gehen. Das ist eigentlich ein super Punkt, den ich im Diskussions-Kapitel aufgreifen kann!
09.08.26	3.0h	Maturaarbeit: Software-Kapitel <i>LaTeX</i>	Verfassen des Kapitels <i>Software-Implementierung</i>. Das Kapitel wurde modular aufgebaut: Für alle Kernfunktionen (Kanalcodierung, Symbol-Mapping, CFO-Korrektur, TDMA-Logik) wird zunächst die mathematisch-logische Funktionsweise erläutert, direkt gefolgt von der Einbindung des jeweiligen Python-Code-Blocks. Zusätzlich wurde die Terminologie im Text präzisiert (z.B. SSSDR-Transceiver mit hybrider Modulation"). Der vollständige Quellcode wurde in den Anhang ausgelagert.

Datum	Zeit	Thema & Ablage	Inhalt
11.08.26	2.5h	Finaler TDMA-Test (300 Pakete) <i>Logfile 098.txt</i>	Das Testskript habe ich für einen grösseren Stresstest angepasst: Das System tauscht nun genau 100 TDMA-Runden mit je 1 Paketen pro Slot (Burst-Modus) aus. Ergebnis: Ein Erfolg auf dem Hp omnibook. Die Log-Dateien zeigen, dass die Frequenzdrift (CFO) sehr genau bei ca. 800 Hz erkannt und korrigiert wird. Der LDPC-Decoder konnte hunderte Bitfehler reparieren, was am Ende zu einer perfekten BER von 0,00 % geführt hat. Das System funktioniert in der Praxis also wirklich so, wie theoretisch geplant. Jedoch wurden nicht alle Pakete erkannt, sondern lediglich 59 von 300.
16.08.26	2.5h	Feldtests an drei Strassen <i>Messprotokolle</i>	Durchführung der finalen Feldtests zur Reichweitenbestimmung an drei unterschiedlichen Strassen. Messung der Signalqualität, Bitfehlerrate (BER) und des Paketverlusts bei verschiedenen Distanzen (20 m, 50 m, 100 m und 200 m). Die gesammelten Daten bilden die Grundlage für die Auswertung der Kanalsymmetrie und Hardware-Stabilität.
18.08.26	3.5h	Inhaltliche Lücken & Quellen <i>LaTeX</i>	Systematisches Überarbeiten des Textes. Schliessen von inhaltlichen Lücken im Diskussions- und Theorieteil. Verfeinerung der Fragestellung, um den roten Faden der Arbeit klarer herauszuarbeiten. Zudem wurden fehlende Fachquellen recherchiert und sauber im Literaturverzeichnis nachtragen.

Datum	Zeit	Thema & Ablage	Inhalt
21.08.26	3.0h	Kriterienabgleich & Promptverzeichnis <i>LaTeX</i>	Abgleich der gesamten Arbeit mit den offiziellen Bewertungskriterien für Maturaarbeiten im Fachbereich Informatik, um sicherzustellen, dass alle formalen und inhaltlichen Vorgaben erfüllt sind. Erstellung und Formatierung des Promptverzeichnisses zur transparenten Dokumentation der verwendeten KI-Hilfsmittel.
24.08.26	4.0h	Sprachlicher Feinschliff & Korrektorat <i>LaTeX</i>	Umfassende sprachliche Umgestaltung und grammatikalische Korrektur des gesamten Dokuments. Sätze wurden umformuliert, um den akademischen Schreibstil zu verbessern. Letzte Formatierungsanpassungen am Layout (Tabellen, Referenzen, Abbildungen) für die finale Abgabeverision vervollständigt.
28.08.26	3.0h	Kriterienabgleich & Layout <i>Informatik-Richtlinien</i>	Detaillierter Abgleich des aktuellen Dokumentenstands mit den offiziellen Bewertungskriterien für Maturaarbeiten im Fachbereich Informatik. Dabei wurde geprüft, ob der Eigenleistungsanteil beim Code und die methodische Vorgehensweise klar genug hervorgehen. Anschliessend grundlegende Anpassungen am Layout vorgenommen, um die formalen Vorgaben (Seitenränder, Zeilenabstand, Platzierung der Verzeichnisse) exakt zu erfüllen.
01.09.26	4.0h	Darstellungen Sprachlicher Feinschliff <i>LaTeX</i>	Systematische Vereinheitlichung aller visuellen Elemente in der Arbeit. Die Tabellen wurden auf ein einheitliches wissenschaftliches Format (mit <code>booktabs</code>) umgestellt und die Beschriftungen der Diagramme und Code-Blöcke standardisiert. Parallel dazu wurde der Text auf akademisches Vokabular geprüft, Schachtelsätze wurden aufgelöst und technische Erklärungen präzisiert.

Datum	Zeit	Thema & Ablage	Inhalt
05.09.26	2.5h	Korrektorat (Gegenlesen) <i>PDF-Entwurf</i>	Die Arbeit wurde von meinem Vater als fachfremde Drittperson gegengelesen. Fokus lag dabei auf der Verständlichkeit des roten Fadens, der Überprüfung von Orthografie und Grammatik sowie dem Aufspüren von Tippfehlern. Das Feedback wurde anschließend besprochen und die entsprechenden Korrekturen direkt in den LaTeX-Quellcode eingepflegt.
07.09.26	1.5h	Finale digitale Überarbeitung <i>VS Code</i>	Letzte Überprüfung der Querverweise, des Literaturverzeichnisses und der Fussnoten auf Vollständigkeit. Die PDF-Datei wurde mehrfach neu kompiliert, um sicherzustellen, dass keine fehlerhaften Seitenumbrüche bei den Abbildungen oder Tabellen entstehen. Das Dokument ist nun inhaltlich und optisch bereit für den baldigen Druck.